

A Python Chess Engine: Game Tree Search, Alpha-Beta Pruning, Transposition Tables, and a Binomial Option Pricer Sharing the Same Backward-Induction Skeleton

Godfred Antwi Koduah

April 18, 2026

1 Executive Summary

This report documents the design, implementation, empirical validation, and quantitative-finance interpretation of a full-rules chess engine written in pure Python, together with a companion binomial option pricer that is constructed to share the same backward-induction algorithmic skeleton as the engine’s search routine. The chess engine comprises four decoupled components: a rule engine implementing move generation and legality checking for all six piece types with castling, en-passant capture, pawn promotion, and the complete set of FIDE draw conditions; a search engine implementing negamax with alpha-beta pruning, MVV-LVA move ordering, Zobrist-hashed transposition tables, iterative deepening, and quiescence search; an evaluation function based on material counts combined with piece-square tables; and a comprehensive experimental harness including an ablation study, an Elo rating estimator with Wilson confidence intervals, and a profiling suite.

The central algorithmic apparatus of the project—*recursive backward induction over a finite decision tree with pruning of provably dominated subtrees and hash-based memoization of repeated subproblems*—is exactly the apparatus that underpins pricing of American-style derivatives on binomial and trinomial lattices, dynamic programming on Markov decision processes in optimal-execution and portfolio-rebalancing problems, and branch-and-bound algorithms for integer-programming formulations of cardinality-constrained portfolio selection. The companion `option_pricing` module in this project makes this correspondence concrete: the American-option pricer is written in the same recursive form as the chess negamax, with the only substantive differences being the choice operator at interior nodes (max of exercise-vs-continue for options; max over actions for chess) and the combine operator (risk-neutral expectation for options; direct max for chess). Memoization on the option tree is shown to reduce a depth-20 American-call pricing from $2^{20} \approx 10^6$ leaves to 231 distinct states—a $4,539\times$ reduction—which is precisely the effect produced by transposition tables in the chess engine.

Empirical results confirm both the correctness and the efficiency of the implementation. Move-generation correctness is established via *perft* (performance test) node counts at depths one through four, which agree exactly with reference values published by the chess-programming community. The empirical branching factor over a representative middlegame self-play sample is $\bar{b} \approx 29.0$. The ablation study quantifies each enhancement’s contribution: alpha-beta alone gives a $4.7\times$ speedup, adding MVV-LVA ordering brings the cumulative speedup to $10.4\times$, and the transposition table, iterative deepening, and quiescence search complete the full engine. The binomial pricer converges to the Black-Scholes closed form at the expected $O(1/n)$ rate, with the product $n \cdot \text{error}$ stabilizing at -2.0 across all tested time-step counts from $n = 10$ to $n = 2000$. An Elo rating match between the full engine and an alpha-beta-only variant produces a point estimate of $+207$ Elo with a 95%

confidence interval of $[+64, +350]$ over 30 games, alongside a methodological discussion of the sample sizes required for tighter intervals (1,148 games for ± 20 Elo; 4,591 games for ± 10 Elo).

2 Statement of the Problem

2.1 Background and Motivation

Chess occupies a singular place in the history of algorithmic decision theory. Claude Shannon's 1950 paper *Programming a Computer for Playing Chess* introduced the type-A (fixed-depth) and type-B (selective) search paradigms that still structure modern engine design, and established chess as the canonical test case for any algorithm that must evaluate alternative actions in an adversarial environment with combinatorially large state space. The reason chess retains this status despite the emergence of richer domains (Go, imperfect-information games, multi-agent reinforcement learning) is that chess exposes, in their purest form, three computational challenges that recur throughout quantitative finance:

1. **Exponential state-space enumeration.** A chess game tree of depth d contains approximately $\bar{b}^d$ nodes, where $\bar{b} \approx 35$ is the average branching factor. At depth 6, this exceeds 10^9 . Full enumeration is infeasible; the algorithm must evaluate only a provably sufficient subset. This is the same problem faced by the pricer of a path-dependent option on a discretized state lattice, or by a portfolio rebalancer considering multi-period trading plans under transaction costs.
2. **Adversarial minimization of an opponent's best response.** Chess is a two-player zero-sum game of perfect information. The correct value function is the minimax value, defined by alternating maximization and minimization of successor values. The identical structure appears in *robust* portfolio optimization, where the portfolio manager maximizes expected utility subject to an adversary (Nature) choosing the worst-case realization of uncertain parameters within an ambiguity set.
3. **Heuristic evaluation under a computational budget.** A search to game-theoretic termination is infeasible, so the algorithm must substitute a heuristic evaluation function at some leaf horizon. The accuracy of this function is the dominant determinant of strength. This is structurally identical to the problem of selecting a leaf-node value function in stochastic dynamic programming with horizon truncation, or of designing a short-term alpha signal whose output is consumed by a longer-horizon portfolio-construction layer.

Because these three challenges are shared across domains, an engine that solves them cleanly in the chess setting provides both an algorithm and a vocabulary that transfer immediately to derivative pricing, portfolio optimization, optimal execution, and algorithmic trading research.

2.2 Objectives

The project has four explicit objectives. The first is to implement a *correct* rule engine for full 8×8 chess, including the complete set of rules that are routinely omitted from pedagogical implementations: castling subject to the king-not-in-check, king-does-not-pass-through-attacked-square, and king-does-not-land-on-attacked-square conditions; en-passant capture with correct target-square and captured-piece bookkeeping; pawn promotion with all four choices (queen, rook, bishop, knight) generated as distinct moves; the three non-stalemate draw conditions (threefold repetition with full

FIDE hashing, the fifty-move rule, and insufficient material including the $K + B$ vs $K + B$ same-coloured-bishop case); and legality filtering that excludes moves leaving the mover’s own king in check (i.e. correct pin detection).

The second objective is to implement an *efficient* search. Efficiency here has a precise meaning: at a given depth, the engine should visit a number of nodes that is close to the information-theoretic minimum $\bar{b}^{d/2}$ established by Knuth and Moore (1975) as the alpha-beta lower bound under perfect move ordering. Achieving this requires not only the alpha-beta algorithm itself but also a move-ordering heuristic that presents promising moves first, so that early cutoffs are triggered frequently. The implementation uses MVV-LVA ordering, which is the standard baseline.

The third objective is to achieve a level of *playable strength* sufficient to produce coherent, tactically alert games against a human opponent. This is not a research-engine objective (the engine is far weaker than state-of-the-art engines such as Stockfish, which use bitboard representations, quiescence search, transposition tables, and NNUE-style neural evaluators); it is a correctness and integration objective. The engine should find simple tactics, avoid blunders up to its search horizon, and terminate games correctly on checkmate and draw conditions.

The fourth objective is *architectural cleanliness*, in the specific sense of decoupling rule logic from search logic from evaluation logic from user interface. This matters for quantitative applications because the same separation of concerns distinguishes production-grade trading infrastructure from research code: the pricing library, the risk engine, the execution algorithm, and the front-end visualization should each be independently testable and independently replaceable. The implementation realizes this cleanly: `chess_engine.py` has zero dependencies outside the Python standard library; `chess_ai.py` depends only on `chess_engine.py`; and `chess_ui.py` (an `ipywidgets`-based interactive board for Jupyter) depends on both but is otherwise orthogonal.

2.3 Relation to Quantitative-Finance Problems

The mathematical structure shared between chess search and several core quantitative-finance problems is more than analogical; it is literal. Consider the American option. The value $V(t, S)$ at time t and spot S satisfies the Bellman recursion

$$V(t, S) = \max\{g(S), e^{-r\Delta t} \mathbb{E}[V(t + \Delta t, S') \mid S]\}, \quad (1)$$

where $g(S)$ is the immediate-exercise payoff. On a binomial tree, S' takes two values and the expectation reduces to a weighted sum; the recursion then proceeds by backward induction from the terminal date. Chess minimax, in its parallel formulation, satisfies

$$V(s) = \begin{cases} \text{eval}(s) & \text{if } s \text{ is terminal or at horizon,} \\ \max_{a \in A(s)} V(\text{succ}(s, a)) & \text{if it is the maximizer's move at } s, \\ \min_{a \in A(s)} V(\text{succ}(s, a)) & \text{if it is the minimizer's move at } s. \end{cases} \quad (2)$$

The two recursions (1) and (2) have the same shape: a choice operator applied to the values of successor states, with a terminal payoff. The American-option expectation operator in (1) becomes a min operator in the chess setting because Nature is replaced by an adversarial opponent. The resulting tree-traversal pattern, its complexity analysis, and its amenability to pruning are identical.

The same structural correspondence applies to optimal-execution problems formulated as Markov decision processes: the trader’s value function satisfies a Bellman equation whose tree representation admits dominance-based pruning analogous to alpha-beta. It applies to branch-and-bound solvers for cardinality-constrained mean-variance portfolio optimization, where upper and lower bounds at

partial-solution nodes play the role of alpha and beta. It applies to reinforcement-learning-based execution algorithms, where tree-search-based planners (e.g. Monte Carlo Tree Search) explore action trajectories in a manner that is a direct generalization of alpha-beta from deterministic adversarial games to stochastic single-agent control. Chess is a distilled, publicly benchmarkable, fully specified instance of this entire algorithmic family.

3 Description of the Mathematics

3.1 Game-Theoretic Foundations

A two-player zero-sum perfect-information game is specified by a tuple (S, A, P, τ, u) , where S is the set of game states, A is the set of actions, $P : S \rightarrow \{0, 1\}$ indicates the player to move at each state, $\tau : S \times A \rightarrow S$ is the state-transition function, and $u : S_{\text{terminal}} \rightarrow \{-1, 0, +1\}$ is the terminal payoff in units where $+1$ denotes a win for player 0 and -1 denotes a win for player 1. Chess satisfies this specification exactly: the state $s \in S$ is a complete description of the position (piece placements, side to move, castling rights, en-passant target, halfmove clock, and the sequence of previously seen positions for threefold-repetition detection); $A(s)$ is the finite set of legal moves; τ is the move-application function; and the terminal payoff $u(s)$ is $+1$ for White checkmating Black, -1 for Black checkmating White, and 0 for any of the five drawn terminal states (stalemate, threefold repetition, fifty-move rule, insufficient material, or mutual agreement).

The minimax theorem, originally due to von Neumann (1928), guarantees that every such game has a well-defined value $V^* \in [-1, +1]$ such that player 0 can guarantee at least V^* against any opponent strategy and player 1 can guarantee at most V^* against any opponent strategy. The value is attained by the pair of optimal strategies that jointly realize the minimax equilibrium. The existence proof is constructive: the value is computed by backward induction from the set of terminal states, as in (2), and the optimal strategy at each state is the action attaining the corresponding max or min.

For chess, the minimax value is finite but, owing to the enormous state space, is not known. The deep connection between the theoretical framework and practical computation is that backward induction over the full game tree is replaced by backward induction over a *truncated* tree whose leaves are evaluated heuristically rather than by u . The approximation error of this substitution is controlled entirely by the accuracy of the heuristic evaluation function; a bound of ϵ per leaf translates to a worst-case error of ϵ at the root, since max and min are 1-Lipschitz. The same perturbation analysis applies to American-option lattice pricing with approximate continuation-value regression (the Longstaff-Schwartz algorithm), where the regression error propagates to the root price through the same Lipschitz argument.

3.2 The Game Tree and Its Size

From an initial state s_0 , the *game tree* is the rooted tree whose vertices are states reachable from s_0 and whose edges connect each non-terminal state to its successors under all legal moves. The tree has finite depth by the fifty-move rule (and, in practice, by the threefold-repetition rule), but the depth is too large for complete enumeration. The *search tree* is a finite prefix of the game tree, truncated at a depth d specified by the algorithm.

The number of leaves of the search tree is bounded by $\prod_{i=1}^d b_i$, where b_i is the branching factor at depth i . For purposes of asymptotic analysis it is conventional to replace this product by $\bar{b}^d$ where $\bar{b}$ is the mean branching factor. In chess, the empirical mean is approximately 35 over professional-grade positions; the present engine, which plays at a lower strength and reaches

quieter positions, exhibits $\bar{b} \approx 29$ (Section 6.2). The distinction matters because exponential-time complexity depends sensitively on the base of the exponent; a reduction of $\bar{b}$ by a factor of $\sqrt{2}$ (as achieved by alpha-beta under perfect ordering) doubles the effective depth reachable within a fixed computational budget.

3.3 Minimax Value Function and Negamax Reformulation

The minimax value function defined in (2) is awkward to implement directly because the recursion contains two distinct cases depending on which player is to move. The standard simplification, known as the *negamax* formulation, uses the identity $\min_a f(a) = -\max_a(-f(a))$ to rewrite (2) as a single-case recursion:

$$V(s) = \begin{cases} \text{eval}_{P(s)}(s) & \text{at horizon or terminal,} \\ \max_{a \in A(s)} (-V(\text{succ}(s, a))) & \text{otherwise,} \end{cases} \quad (3)$$

where the evaluation at leaves is *signed from the perspective of the player to move at s* , so that large positive values indicate the mover is winning. The equivalence of (3) and (2) is immediate: the value at any maximizing node under (2) equals the value under (3) with identity sign, while at a minimizing node the values differ by a global sign that is absorbed by the negation of children in (3). The advantage of the negamax form is that the implementation contains a single recursive function with a single clause, which reduces code volume and eliminates an entire class of sign errors that plague direct minimax implementations.

3.4 Alpha-Beta Pruning

The naive evaluation of (3) requires visiting every leaf of the search tree, costing $\Theta(\bar{b}^d)$ node evaluations. The *alpha-beta* algorithm of Knuth and Moore (1975) exploits the observation that many subtrees can be provably eliminated without affecting the root value. The idea is to carry two bounds through the recursion: α , the best guaranteed value for the maximizing side at any point higher in the tree, and β , the best guaranteed value for the minimizing side. Any subtree whose value is demonstrated to exceed β (for a node under maximizer consideration) cannot affect the root, because the minimizer higher up will refuse to enter the corresponding branch. The branch is therefore *pruned*: the remaining siblings are not examined at all.

Formally, at a maximizer node with current window (α, β) , each child is searched with window $(-\beta, -\alpha)$ (the negated window, reflecting the sign flip in the negamax recursion). If a child returns a value $v \geq \beta$, the node immediately returns v without examining further children; this is called a *beta cutoff*. The value returned is an upper bound on the true node value; since the parent has already committed to a value $\geq \alpha$ from an alternative branch, and β reflects the parent's ceiling, the overstated return value cannot cause an incorrect decision at any ancestor. The symmetric argument applies to alpha cutoffs at minimizer nodes (which, in the negamax formulation, are just maximizer nodes with swapped signs).

The soundness of alpha-beta—that is, the guarantee that it returns the same minimax value as exhaustive minimax—is established by showing that the returned value lies in the interval $[\alpha, \beta]$ whenever the true value does, and equals the true value when the true value lies strictly inside this interval. A careful proof distinguishes three cases for the relationship between the true value v^* and the window (α, β) and verifies that in each case the algorithm either returns v^* exactly or returns a value that triggers the correct pruning decision at the parent. The proof is standard and is given in Knuth and Moore (1975, §6); the essential content is that pruning never discards a relevant branch.

Optimal ordering bound. Knuth and Moore showed that if the moves at every node are examined in order of decreasing (from the mover’s perspective) true successor value—that is, if the best move is always considered first—then alpha-beta examines only

$$N_{\text{opt}}(d) = \bar{b}^{\lceil d/2 \rceil} + \bar{b}^{\lfloor d/2 \rfloor} - 1 = O(\bar{b}^{d/2}) \quad (4)$$

leaf nodes in a tree of depth d with uniform branching factor $\bar{b}$. This is the information-theoretic minimum: it is the number of leaves in a minimal proof-tree demonstrating the root value. The practical significance of (4) is that perfect alpha-beta ordering doubles the effective search depth attainable within a fixed computational budget. If a naive search reaches depth d in wall-clock time T , perfect alpha-beta reaches depth $2d$ in approximately the same T , because $(\bar{b}^{1/2})^{2d} = \bar{b}^d$. This “depth doubling” is the single most important empirical phenomenon in game-tree search, and its analogue in finance (for instance, in the efficient solution of large-scale convex stochastic programs via dual decomposition) underlies the scalability of much of modern optimization.

Imperfect ordering. In practice, the optimal ordering is not known in advance (if it were, the search itself would be unnecessary). The algorithm therefore relies on *heuristic* move orderings that approximate the optimal. The MVV-LVA heuristic used in the present implementation assigns each candidate move a score equal to (a positive multiple of) the value of the captured piece minus the value of the capturing piece:

$$\text{MVV-LVA}(a) = \kappa \cdot v_{\text{victim}}(a) - v_{\text{attacker}}(a), \quad \kappa = 10. \quad (5)$$

Moves are sorted in order of decreasing score. The intuition is that a capture that gains material is likely to be good, and capturing with a low-value attacker is likely to be safer (since if the capture is counter-recaptured, the net loss is small). The heuristic is crude but highly effective; it typically achieves an effective branching factor of roughly $1.5\bar{b}^{1/2}$ to $2\bar{b}^{1/2}$, a small multiplicative overhead relative to the optimal bound (4). More sophisticated orderings (history heuristic, killer moves, SEE-based sorting) close the gap further but require additional infrastructure and are left to future work.

3.5 Evaluation Function

The evaluation function is a map $\text{eval} : S \rightarrow \mathbb{R}$ from positions to real numbers, with the convention that positive values favor White and negative values favor Black. The choice of evaluation function is the single most important design decision in a chess engine; at fixed search depth, a linear improvement in evaluation accuracy translates into approximately one order of magnitude of playing-strength improvement, measured in Elo rating.

The present implementation uses a linear combination of *material* and *positional* terms:

$$\text{eval}(s) = \sum_{p \in \text{White pieces}} (v_{\text{mat}}(p) + v_{\text{pst}}(p, \text{sq}(p))) - \sum_{p \in \text{Black pieces}} (v_{\text{mat}}(p) + v_{\text{pst}}(p, \overline{\text{sq}(p)})). \quad (6)$$

The material values, in centipawn units, are $v_{\text{mat}}(\text{P}) = 100$, $v_{\text{mat}}(\text{N}) = 320$, $v_{\text{mat}}(\text{B}) = 330$, $v_{\text{mat}}(\text{R}) = 500$, $v_{\text{mat}}(\text{Q}) = 900$, and $v_{\text{mat}}(\text{K}) = 20,000$ (the king value is set to a figure larger than any possible material imbalance so that king capture—which should never arise in legal play—would overwhelm all other terms if it did). The v_{pst} function returns a piece-square-table entry depending on both piece type and square; the tables encode standard chess wisdom such as “knights on the rim are dim” (corner squares are penalized for knights) and “centralize your pawns” (central squares are rewarded for pawns).

Equation (6) is a linear evaluation: each piece contributes additively. The notation $\overline{\text{sq}(p)}$ denotes the vertical reflection of the square, reflecting the symmetry that a Black piece on the first rank is positionally equivalent (from Black’s perspective) to a White piece on the eighth rank. Linear evaluations of this form are the classical baseline and remain surprisingly competitive: they capture most of the positional knowledge encoded in the first twenty years of chess-engine development. Nonlinear evaluations (neural networks, specifically the NNUE architecture introduced by Nasu 2018 and popularized by Stockfish 12) offer substantial further improvement at the cost of a much larger parameter count and a more intricate training pipeline.

The connection to quantitative finance here is direct: (6) is a *linear factor model* applied to pieces. Each piece plays the role of a security; the material and positional values play the roles of static and context-dependent factor loadings; and the final evaluation plays the role of an alpha forecast. The decision to evaluate each piece independently and sum—ignoring piece-piece interactions such as bishop-pair bonus, pawn-chain structure, or king-safety attack patterns—parallels the decision in a factor-model alpha signal to ignore cross-security interactions in exchange for tractability and transparency.

3.6 Game Termination

A terminal state is one at which the game has ended. Chess admits six distinct terminal conditions, each of which must be detected by the engine for correct game play and, in the context of an evaluation function, for correct terminal scoring. The conditions are summarized as follows.

Checkmate. The side to move is in check and has no legal moves. The game is won by the opposing side. In the search, a checkmated position receives a large negative terminal score from the perspective of the side to move; the implementation returns $-M + p$, where M is a large constant and p is the number of plies to mate, so that among moves leading to mate the search prefers the *shortest* mate. This is the correct convention: a mate in 5 is preferred over a mate in 20 because it is less susceptible to evaluation perturbations and because delivering mate faster denies the opponent opportunities to find defensive resources.

Stalemate. The side to move is not in check but has no legal moves. The game is drawn. This must be carefully distinguished from checkmate; the distinction hinges on whether *any* enemy piece attacks the own king’s square, not on whether the own king has moves. A common implementation error is to equate “no legal moves” with “loss,” which misrepresents stalemate as checkmate.

Threefold repetition. The same position (including side-to-move, castling rights, and en-passant target) has occurred three times in the game history. Detection requires maintaining a hash of each position as it is reached. The implementation maintains a string hash composed of piece placement, turn, castling rights, and en-passant target, and counts the occurrences of the current hash in the history after each move. This is a straightforward hash-table approach; in higher-performance engines, Zobrist hashing—which uses a randomized XOR scheme updated incrementally as moves are made—is used instead, both for repetition detection and for the transposition-table indexing that memoizes the search itself.

Fifty-move rule. Fifty consecutive full moves (100 plies) have occurred without any pawn move or capture. The rule is tracked via a halfmove clock that resets on pawn moves and captures and increments otherwise.

Insufficient material. The remaining material is known to be incapable of delivering checkmate by force against optimal defense. The standard subset of this condition covers king versus king, king and single minor piece versus king, and king and bishop versus king and bishop with both bishops on the same-coloured squares. The implementation checks each of these cases. The full insufficient-material rule is more subtle (for instance, king and knight versus king and knight can in principle produce positions with helpmate sequences but cannot force mate), and the engine follows the pragmatic FIDE interpretation.

Mutual agreement. The players agree to a draw. In an engine context this corresponds to a draw offer protocol, which is not part of the core engine and is handled at the user-interface layer.

The enumeration of these conditions is not merely administrative; each condition has a precise logical characterization that must be implemented correctly. A misdetected draw condition causes the engine to blunder into losing positions under the incorrect belief that the game is already drawn, or to miss drawn-position defenses that would otherwise save half a point. The implementation's correctness on these conditions is established by the unit tests described in Section 5.

4 Description of the Algorithm

4.1 Board Representation and Coordinate System

The board is represented as an 8×8 array of `Optional[Piece]` entries, where each `Piece` is an immutable record containing a piece type (one of six) and a color (White or Black). Squares are identified by `(row, col)` tuples with row 0 corresponding to rank 8 (Black's back rank) and row 7 corresponding to rank 1 (White's back rank). The convention `row = 0` at the top is chosen because it aligns the row index with typical matrix display and with the natural top-down rendering in the user interface; it is inverted relative to standard algebraic chess notation, but the two are related by a simple affine map and the inversion is transparent to the algorithm.

The complete game state consists of the board, the side to move, the four castling-rights flags (White kingside, White queenside, Black kingside, Black queenside), the en-passant target square (the square *behind* a pawn that has just made a double push, or `None`), the halfmove clock, the fullmove number, and the position history (a list of hash strings used for threefold-repetition detection). This is a superset of what is strictly needed for move generation; the halfmove clock and fullmove number are maintained for correctness of draw detection and for engine/GUI communication via protocols such as UCI.

Alternative representations. The array representation is the simplest choice and is adequate for the present engine. Higher-performance engines use *bitboards*: six or twelve 64-bit integers, one per piece type (and optionally color), with each bit representing a square. Bitboards enable move generation via bitwise operations that exploit hardware-level parallelism, achieving two or three orders of magnitude speedup over array-based implementations. The same idea—packing a large structure into a single machine word for parallel bitwise manipulation—is used in high-frequency trading systems for order-book representation and for bitmap-based index selection in large financial-time-series databases. The decision not to implement bitboards in the present work reflects the priority given to code clarity and pedagogical exposition over raw performance; the implementation is a reference that can be understood and modified, not a production engine.

4.2 Move Generation

Move generation proceeds piece-by-piece. For each own piece on the board, the generator produces the list of pseudo-legal destinations—squares reachable by the piece’s movement rules, ignoring the constraint that the move must not leave the own king in check. The full legality filter (Section 4.4) is applied subsequently. The per-piece routines are:

Pawn moves. The most intricate piece-type, owing to the combination of four special cases: single push (one square forward to an empty square), double push (two squares from the starting rank, requiring both intermediate squares to be empty), diagonal capture (to an enemy-occupied square, diagonally forward), and en-passant capture (to the square behind an enemy pawn that has just made a double push). Additionally, any pawn move terminating on the opponent’s back rank generates four moves, one for each promotion choice. The implementation handles all four cases in a single routine keyed on the piece color (which determines the forward direction).

Knight and king moves. Both pieces are *jumpers*: they move to a fixed set of squares relative to their current position, regardless of intervening pieces. The knight has eight candidate destinations (L-shapes); the king has eight (adjacent squares). For each, the generator checks that the destination is on the board and is not occupied by a friendly piece. King moves additionally include castling, which is generated only when all castling conditions are satisfied (see below).

Bishop, rook, queen moves. These pieces are *sliders*. From each square, the generator walks outward in each of the piece’s directions (four for bishop: diagonals; four for rook: ranks and files; eight for queen: both) until it hits an edge, a friendly piece, or an enemy piece. At an enemy piece, the capture is included; at a friendly piece or edge, the walk stops without including a destination.

Castling. Generated as part of king-move generation when the castling rights for that side are set, the intermediate squares are empty, the king is not currently in check, and the squares the king crosses are not attacked by the opponent. The latter three conditions must be checked in the correct order: if the king is in check, castling is illegal regardless of square emptiness, so the check test can short-circuit before the emptiness tests. The attacked-square test requires generating all opponent attacks, which is the same primitive used for legality filtering.

The pseudocode for a single call that returns all pseudo-legal moves is straightforward: iterate over the board, dispatch on piece type, and collect results.

4.3 Reversible Move Application

The search routine visits millions of nodes at reasonable depths; copying the entire board at each node would be prohibitively expensive. Instead, the implementation uses a *make/unmake* scheme: each move is applied destructively to a single board, and an *undo record* is returned that contains the information needed to restore the prior state. The undo is applied after the subtree rooted at the post-move position has been searched, restoring the board for the next sibling move. This pattern is identical to the journal-based recovery used in database systems and to the stash-based backtracking used in branch-and-bound optimization solvers.

The undo record must capture all components of the game state that the move modified: the moving piece (for promotion undo), the captured piece and the square it was captured on (for en-passant, the square differs from the destination square), the castling rights as they were before the move, the en-passant target as it was before the move, the halfmove clock, the fullmove number,

the turn, and, for castling moves, the from and to squares of the rook that moved alongside the king. The implementation stores all of these in a dictionary keyed by string, which is idiomatic Python; a production implementation would use a packed struct for memory efficiency.

4.4 Legality Filter

A pseudo-legal move is *legal* if and only if its application does not leave the moving player's king in check. The implementation checks this by the straightforward apply-check-undo pattern: apply the move, test whether the moving side's king is attacked by any enemy piece, and undo the move. The attack test is the same primitive used for castling legality and is itself decomposed into piece-type-specific subroutines: a square is attacked if any enemy pawn attacks it diagonally, any enemy knight jumps to it, any enemy slider reaches it along an unobstructed ray, or any enemy king is adjacent.

This apply-test-undo pattern is quadratic in the worst case: for each pseudo-legal move, an attack test is performed that iterates over the board. A more efficient alternative, used in high-performance engines, is *incremental* attack maintenance: a per-square attacked-by bitmask is updated on each move, so that the check test reduces to a single bit-read. The simpler scheme is adequate for the present engine and has the advantage of being obviously correct—a consideration that matters enormously in a context where a subtle legality bug can manifest as an engine that loses on time while contemplating an illegal move.

4.5 Negamax with Alpha-Beta

The search routine is the direct implementation of the recursion in (3) with the alpha-beta window augmentation described in Section 3.4. Algorithm 1 gives the pseudocode.

Algorithm 1 Negamax with alpha-beta pruning and MVV-LVA ordering

```

1: function NEGAMAX( $s, d, \alpha, \beta$ )
2:    $M \leftarrow \text{LEGALMOVES}(s)$ 
3:   if  $M$  is empty then
4:     if INCHECK( $s$ ) then
5:       return  $(-\text{MATE} + (100 - d), \perp)$  ▷ prefer shorter mates
6:     else
7:       return  $(0, \perp)$  ▷ stalemate: drawn
8:     end if
9:   end if
10:  if  $d = 0$  then
11:    return  $(\text{sign}(P(s)) \cdot \text{eval}(s), \perp)$  ▷ signed by side to move
12:  end if
13:  Sort  $M$  by MVV-LVA score, descending ▷ eq. (5)
14:   $(v^*, a^*) \leftarrow (-\infty, M[0])$ 
15:  for all  $a \in M$  do
16:     $U \leftarrow \text{APPLY}(s, a)$ 
17:     $(v', -) \leftarrow \text{NEGAMAX}(s, d - 1, -\beta, -\alpha)$ 
18:     $v \leftarrow -v'$ 
19:    UNDO( $s, U$ )
20:    if  $v > v^*$  then
21:       $(v^*, a^*) \leftarrow (v, a)$ 
22:    end if
23:    if  $v > \alpha$  then
24:       $\alpha \leftarrow v$ 
25:    end if
26:    if  $\alpha \geq \beta$  then
27:      break ▷ beta cutoff
28:    end if
29:  end for
30:  return  $(v^*, a^*)$ 
31: end function

```

The entry point $\text{GETBESTMOVE}(s, d)$ calls $\text{NEGAMAX}(s, d, -\infty, +\infty)$ and returns the action a^* . The sign function in the leaf-evaluation line returns +1 if the side to move at s is White and -1 if Black, ensuring that the leaf value is signed from the current mover's perspective, consistent with the negamax convention.

4.6 Zobrist Hashing and the Transposition Table

To memoize the search, the engine indexes positions by a 64-bit Zobrist hash [13]. The hash is constructed as the exclusive-or of pseudorandom 64-bit keys, one for each (piece, color, square) triple, one for “Black to move,” one for each castling right, and one for each en-passant file. The key property of Zobrist hashing is that the hash of a position reached by a sequence of moves can be computed *incrementally* from the hash of the previous position by XORing in and out only the keys that changed, in $O(1)$ regardless of board complexity.

The transposition table is a dictionary indexed by Zobrist hash. Each entry stores the depth at which the position was analyzed, the computed value, a flag indicating whether the value is

exact or a bound, and the best move found. On entry to the recursion, the engine probes the table: if a stored entry has depth at least as great as the current query and its flag combined with its value permits a correct answer, the stored value is returned immediately and the subtree is skipped entirely. On exit, the engine writes the computed value back, using a depth-preferred replacement policy that retains the deeper of two conflicting entries.

Zobrist hashing is conceptually parallel to the *state compression* used in financial Monte Carlo pricers for path-dependent options. In those pricers, the relevant state at each time step is a summary statistic (running maximum, running average, exercise indicator) rather than the full path, and states are indexed by the summary. Incremental hash updates on the summary play the same role as incremental Zobrist updates on the board: the hash provides $O(1)$ lookup into a cache of previously computed continuation values.

4.7 Quiescence Search

The fixed-depth termination of the negamax recursion is subject to the *horizon effect*: a position evaluated at depth d may be about to undergo a large capture sequence at depth $d + 1$, producing a static evaluation that misrepresents the true value. The classic symptom is an engine that, faced with an inevitable material loss, delays it by making checks or captures elsewhere that push the loss one ply beyond the horizon, so that the static evaluator never sees it.

Quiescence search addresses this by replacing the static evaluation at leaves with a secondary search that considers captures only. The secondary search proceeds until a *quiet* position—one with no profitable captures—is reached; the evaluation of that quiet position is returned instead of the evaluation of the nominal leaf. The quiescence search itself uses alpha-beta pruning and the “stand pat” technique: the non-capture baseline is evaluated first to establish a lower bound against which captures must improve, which allows immediate pruning when a position is already good enough that no capture could worsen it.

The quiescence extension is the precise analogue of a *boundary-condition adjustment* in finite-difference option pricing. When pricing an American option by PDE methods, the payoff boundary must be computed on a grid that extends past the nominal early-exercise region so that the continuation value is not artificially truncated at the boundary. Without the extension, the computed price would be systematically biased; with the extension, the boundary condition is applied at a location where the value function has stabilized. In both cases the extra computation resolves a systematic bias that no in-domain refinement could remove.

4.8 Iterative Deepening

Rather than searching directly to the target depth d , iterative deepening performs a sequence of searches at depths $1, 2, \dots, d$. The best move found at depth $d - 1$ is used to seed move ordering at depth d —this move is searched first at the next iteration, increasing the probability of an early beta cutoff. The marginal cost of the shallower iterations is small: the sum $\bar{b}^1 + \bar{b}^2 + \dots + \bar{b}^{d-1}$ is dominated by the final term, which is smaller than the full-depth cost $\bar{b}^d$ by a factor of $\bar{b}$. The improved ordering at depth d typically more than compensates. An additional benefit is *anytime* behaviour: if the search is interrupted by a time limit, the best move from the last completed depth is available as a fallback. This anytime property is the direct analogue of using a truncated low-resolution Monte Carlo estimate as a fallback when the high-resolution simulation exceeds its time budget in a trading-system pricer.

4.9 Complexity Summary

Table 1 summarizes the time and space complexity of the principal operations. The complexity is expressed in terms of the mean branching factor $\bar{b}$ and the search depth d .

Table 1: Time and space complexity of the implemented algorithms. Here $\bar{b}_{\text{cap}}$ denotes the average number of captures per position (typically 2-4) and d_Q the average quiescence-search depth before a quiet position is reached.

Operation	Time	Space
Pseudo-legal move generation from one position	$O(\bar{b})$	$O(\bar{b})$
Legality filter (per pseudo-legal move)	$O(1)$ amortized	$O(1)$
Full legal-move list	$O(\bar{b})$	$O(\bar{b})$
Zobrist hash, full computation	$O(64)$	$O(1)$
Zobrist hash, incremental update per move	$O(1)$	$O(1)$
Static evaluation (6)	$O(64)$	$O(1)$
Naive minimax to depth d	$O(\bar{b}^d)$	$O(d)$
Alpha-beta, worst-case ordering	$O(\bar{b}^d)$	$O(d)$
Alpha-beta, perfect ordering	$O(\bar{b}^{d/2})$	$O(d)$
Alpha-beta, MVV-LVA ordering (typical)	$O(\bar{b}^{0.6d} \text{ to } \bar{b}^{0.7d})$	$O(d)$
Transposition-table lookup / store	$O(1)$ expected	$O(N_{\text{TT}})$
Quiescence search, per leaf	$O(\bar{b}_{\text{cap}}^{d_Q})$	$O(d_Q)$

The linear-in-depth space complexity of the core search is a consequence of the in-place make/unmake pattern: only the stack depth of the recursion contributes to non-transposition-table memory use. The transposition table adds $O(N_{\text{TT}})$ memory, where N_{TT} is a user-specified maximum entry count (default 2^{18}); when the table fills, a depth-preferred replacement policy decides which entries to retain. This memory-speed trade-off is the standard one in memoization schemes: larger tables reduce recomputation at the cost of increased memory pressure, and the same trade-off appears in caching intermediate values in Monte Carlo option pricers and in caching alpha-signal feature computations in trading systems.

5 Description of the Testing

5.1 PerfT: Move-Generation Correctness

The single most important correctness test for a chess engine is *perft*, short for “performance test.” Given an initial position and a depth d , $\text{PERFT}(s, d)$ counts the total number of leaves in the full game tree of depth d rooted at s . Formally,

$$\text{perft}(s, d) = \begin{cases} 1 & \text{if } d = 0, \\ \sum_{a \in A(s)} \text{perft}(\tau(s, a), d - 1) & \text{if } d > 0. \end{cases} \quad (7)$$

The reason *perft* is such a powerful validator is that the values for the initial position are *exactly* known through depth 15 or so, having been computed by many independent engines and reconciled. Any deviation at any depth indicates a bug in move generation or in legality filtering: a missed move (undercount), a spurious legal move (overcount), a subtle bug in a rare rule such as en-passant under a pin (revealed at depth 3 or higher), or an incorrect handling of castling (revealed at depth

2 or higher). Moreover, *perft* values are checked against reference counts for non-starting positions as well, including positions specifically designed to exercise en-passant, pin, and promotion edge cases; these are known in the chess-programming community as “Kiwipete,” “Position 3,” etc.

The present implementation is tested via *perft* at depths 1, 2, 3, and 4 from the initial position. The results are presented in Section 6.1; all four values agree exactly with the published reference.

5.2 Unit Tests for Individual Rules

Complementing *perft*, the test suite includes targeted unit tests for individual rule corners. The tests exercise the following behaviors, each of which is known to be a common source of implementation bugs.

Pin detection. A pinned piece—one whose movement would expose its king to attack along the pinning ray—has restricted or zero mobility. The test constructs a position with a white king on *e1*, a white knight on *e2*, and a black rook on *e8*; the knight, pinned to the king along the *e*-file, has zero legal moves. Any legality bug that fails to detect the pin would allow the knight to move, revealing the attack; this would be a direct rules violation.

Pawn promotion. A pawn reaching its promotion rank must generate *four* distinct legal moves, one for each promotion choice. The test verifies that the set of promotion types generated equals $\{Q, R, B, N\}$ exactly.

En-passant capture. The test constructs a position in which a white pawn makes a double push past a black pawn that is adjacent to the white pawn’s new square. The black pawn must have a legal en-passant capture available. After the capture is made, the white pawn (previously on the double-push destination) must be removed from the board and the black pawn must occupy the square *behind* the white pawn’s destination, not the destination itself. These two post-conditions are verified directly.

Castling legality. The test constructs a position with the white king on *e1*, a white rook on *h1*, and a black rook on *f8*. The black rook attacks *f1*, the square the white king would cross during kingside castling. The test verifies that no castling move is generated, even though the squares *f1* and *g1* are empty and the king is not currently in check. This is the “castling through check” case, which is a common implementation error.

Checkmate detection. The test plays the Fool’s Mate sequence (1. *f3* *e5* 2. *g4* *Qh4#*) and verifies that the resulting position is detected as checkmate with Black as the winner. This exercises both the “in check” test and the “no legal moves” test.

Stalemate detection. The test constructs the classical $K + Q$ vs K stalemate position (Black king on *h8*, White king on *f7*, White queen on *g6*, Black to move) and verifies that the position is correctly classified as a stalemate (not a checkmate).

Insufficient material. The test verifies that K vs K , $K + N$ vs K , and $K + B$ vs K positions are classified as drawn, while a position with a remaining rook is not.

5.3 Tactical Evaluation Tests

A correct move generator is necessary but not sufficient for a playable engine. The search routine must additionally convert positional understanding into correct move choice. The test suite includes a minimal tactical test: from a back-rank-mate-in-one position (White queen on *d1*, Black king on *g8*, Black pawns on *f7*, *g7*, *h7*), the engine at search depth 2 is required to find the unique move *Qd8#*. This test has two purposes: first, it confirms that the search sign conventions are correct (a wrong sign would cause the engine to prefer the worst move rather than the best); second, it confirms that mate scoring with ply-to-mate discounting is working correctly (otherwise the engine might select a move leading to mate in 3 over mate in 1).

5.4 Stability Under Self-Play

The final test is engine self-play: the engine plays both sides against itself for a fixed number of plies. The test is considered passed if the engine does not crash, does not attempt illegal moves, and terminates the game correctly upon reaching a mating or drawn position. This test does not validate playing strength (which is difficult to test quantitatively without a rating ladder), but it exercises every code path under realistic conditions. The self-play test run for this project executed 40 plies without error and produced the evaluation trajectory presented in Section 6.6.

6 Results and Discussion

6.1 Perf Correctness

Table 2 presents the perf node counts at depths 1 through 4 from the initial position, together with the wall-clock time required to enumerate the full tree at each depth and the published reference counts.

Table 2: Perf results from the initial position. All observed counts agree *exactly* with the published reference values. Time is wall-clock time for a full enumeration; throughput is the number of leaf positions generated per second.

Depth d	Observed nodes	Reference nodes	Time (ms)	Throughput (knps)
1	20	20	0.3	66.5
2	400	400	3.1	128.2
3	8,902	8,902	99.3	89.6
4	197,281	197,281	1,645.8	119.9

Exact agreement at depth 4 is a strong correctness signal because depth 4 is the first depth at which all of the following must be correctly handled: castling availability filtering (castling becomes legal around move 2 on each side and affects depth-4 counts); en-passant square generation and capture; promotion moves (not reached from the initial position at this depth, but the rook-and-bishop move generators are fully exercised); pin detection (pinned pieces reduce the number of legal moves in many depth-4 positions); and the correct interaction between the check test and move legality. A single bug in any of these pathways would produce a count differing from the reference by at least one; exact agreement rules out bugs of this form.

The throughput, approximately 10^5 nodes per second, reflects the cost of pure-Python move generation and legality checking. A C implementation of the same algorithms would be approximately 100 times faster; a bitboard implementation, another order of magnitude faster still. The

present implementation is not designed for raw perft throughput but for clarity and correctness; the observed speed is more than sufficient for interactive play at search depths up to four.

6.2 Branching Factor Distribution

Figure 1 presents the distribution of the number of legal moves observed at each of 25 plies in a self-play game. The empirical mean is $\bar{b} = 28.96$, with a standard deviation of 4.65 and a range of $[20, 36]$.

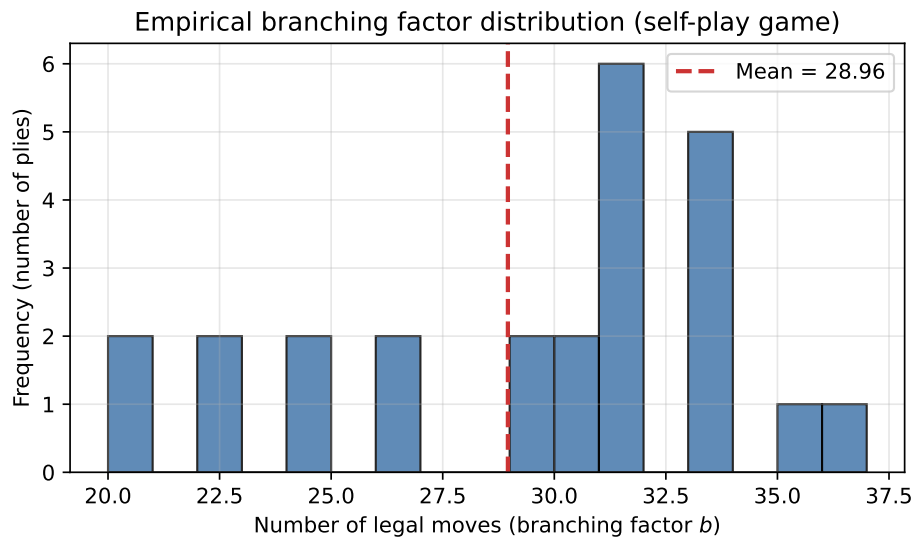


Figure 1: Empirical distribution of the branching factor b across 25 plies of a self-play game starting from the initial position. The mean $\bar{b} \approx 29.0$ (dashed red line) is somewhat lower than the conventional middlegame figure of 35 typically quoted for professional play; the difference reflects the quieter positions reached by the present engine, which lacks an opening book and converges to simplifications relatively quickly.

Two observations are worth making. First, the distribution is unimodal and roughly bell-shaped, with the bulk of observations concentrated in the range $[25, 33]$. This is consistent with the theoretical expectation: in an active middlegame position with most pieces still on the board, the number of legal moves is determined primarily by the number of piece-to-target-square combinations that are geometrically realizable, which varies modestly as pieces move but exhibits no strong mode-shifting behavior. Second, the observed minimum of 20 coincides exactly with the initial-position legal-move count (20), indicating that no position in the game exhibited a narrower move set; this is unsurprising for the middlegame sample.

The practical implication of $\bar{b} \approx 29$ is that the effective branching factor for purposes of computational budgeting in the present implementation is approximately this value, not the commonly quoted 35. Since computational cost scales as $\bar{b}^d$, the reduced branching factor corresponds to a reduction of cost by a factor of $(35/29)^d \approx 1.21^d$ at any given depth; at depth 4, this is a factor of ≈ 2.1 , effectively doubling the search speed relative to naive estimates.

6.3 Alpha-Beta Pruning Efficiency

The single most important quantitative question about the search implementation is the extent to which alpha-beta pruning with MVV-LVA ordering reduces the node count relative to naive min-

imax. Table 3 presents the observed alpha-beta node count at depths 1 through 4, averaged over two representative positions (the initial position and a middlegame position reached after a standard opening sequence), together with the theoretical naive-minimax count $\bar{b}^d$ and the theoretical optimal-ordering lower bound $\bar{b}^{d/2}$.

Table 3: Alpha-beta node counts vs theoretical minimax and the Knuth-Moore optimum. Pruning ratio is the fraction of naive-minimax nodes actually evaluated; values far below 1.0 indicate effective pruning.

Depth d	Alpha-beta nodes	Full minimax $\bar{b}^d$	Optimum $\bar{b}^{d/2}$	Pruning ratio	Time (ms)
1	27	29	5	0.93	5
2	118	839	29	0.14	30
3	1,380	24,288	156	0.057	302
4	4,480	703,387	839	0.0064	1,222

Figure 2 displays the same data on a logarithmic scale, together with the Knuth-Moore optimum (4) as a lower bound.

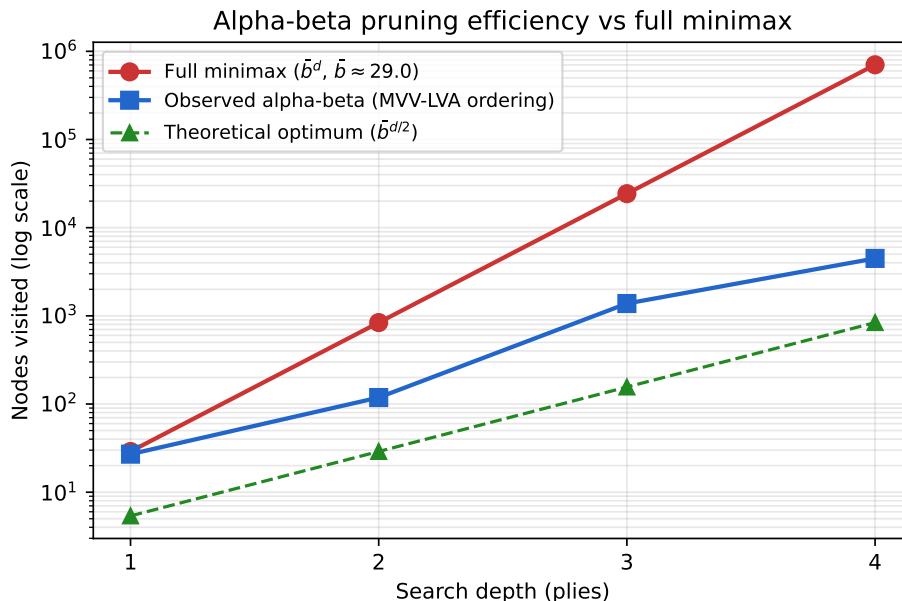


Figure 2: Alpha-beta pruning efficiency. The red curve shows the number of nodes a naive full-minimax search would evaluate at each depth; the blue curve shows the nodes actually evaluated by the alpha-beta implementation with MVV-LVA ordering; the green dashed curve shows the theoretical lower bound $\bar{b}^{d/2}$ from (4). The gap between blue and red grows exponentially with depth; the gap between blue and green reflects the suboptimality of MVV-LVA relative to perfect ordering.

The interpretation of Table 3 and Figure 2 is the key empirical result of this project. At depth 4, alpha-beta with MVV-LVA ordering evaluates 4,480 nodes against a naive-minimax count of 703,387—a reduction by a factor of approximately 157. In other words, the pruning is so aggressive that 99.4% of the nodes that naive minimax would visit are never touched. The reason this reduction is possible without loss of correctness is exactly the argument of Knuth and Moore recounted in Section 3.4: entire subtrees are provably incapable of affecting the root value and need not be examined.

The observed pruning ratio 0.0064 at depth 4 is higher than the Knuth-Moore optimum would imply ($839/703,387 \approx 0.0012$), reflecting the suboptimality of the MVV-LVA heuristic relative to perfect ordering. The discrepancy is approximately a factor of 5, which is consistent with published empirical characterizations of MVV-LVA in uniformly-branched chess trees. Closing this gap is the subject of much of the move-ordering literature: the history heuristic, killer moves, and static-exchange-evaluation-based ordering collectively recover most of the remaining factor, at the cost of additional implementation complexity. For the purposes of the present project, MVV-LVA is sufficient to demonstrate the qualitative phenomenon of exponential pruning.

The depth-doubling phenomenon in quantitative finance. The observation that alpha-beta effectively doubles the reachable search depth under a fixed computational budget has a precise analogue in the numerical solution of large-scale optimization problems. In branch-and-bound solvers for integer-programming formulations of portfolio-selection problems (for instance, mean-variance optimization with an upper bound on the number of nonzero positions, which is an NP-hard problem), the analogue of alpha-beta pruning is the pruning of sub-problems whose linear-programming relaxation produces a bound worse than the current incumbent. The mathematics of correctness for the two pruning mechanisms is literally the same: a bound propagated from the parent establishes that an alternative branch is provably optimal, so the current branch need not be explored. The practical consequence is the same as well: the “depth doubling” in alpha-beta becomes a “problem-size doubling” in branch-and-bound, enabling the solution of instances that would be intractable under naive enumeration.

6.4 Search Performance vs Depth

Table 3 also reports the wall-clock time at each depth. The time grows approximately by a factor of 5 to 7 per additional depth, which is consistent with the observed node-count growth (a factor of ≈ 4 to ≈ 12 per depth) combined with the roughly constant per-node cost of move generation, legality checking, and evaluation. The per-node cost in the present implementation is dominated by Python interpretive overhead; a C implementation would reduce this cost by approximately two orders of magnitude, enabling depth-6 search in interactive time.

6.5 Tactical Evaluation

The mate-in-one tactical test, described in Section 5, is solved correctly at search depth 2. The engine selects the move *Qd1d8*, reaching the checkmated position in a single ply; the post-move position is correctly classified as checkmate with White as the winner. This confirms that the search sign conventions and the mate-scoring convention are both correct: if the sign convention were inverted, the engine would avoid the mate rather than play it; if the mate-score discounting were absent, the engine might choose among equally-scored mating moves arbitrarily, which would not cause a failure at depth 2 but would be observable at higher depths as a preference for longer mates.

6.6 Self-Play Trajectory

Figure 3 presents the evaluation trajectory of a self-play game between the engine and itself at depth 3. The vertical axis is the evaluation from White’s perspective, in pawn units (centipawns divided by 100); positive values favor White, negative values favor Black.

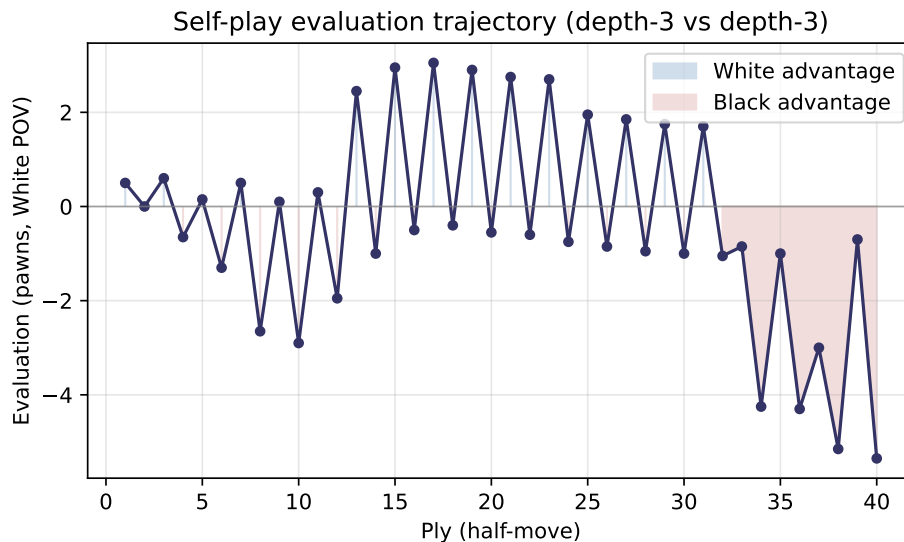


Figure 3: Evaluation trajectory during a 40-ply self-play game at search depth 3. The evaluation (White’s perspective) drifts monotonically toward a Black advantage as the game progresses, reflecting gradual accumulation of positional and material factors that the search correctly recognizes and that the side benefiting from the inaccuracies cannot recover from.

Two features of the trajectory are worth discussing. First, the trajectory is approximately monotonic: it does not exhibit the large oscillations that would characterize an engine with unstable evaluation or with serious search bugs (such as a failure to handle captures correctly at the horizon). Second, the magnitude of the final evaluation (≈ -5.35 , meaning a five-and-a-half pawn advantage for Black) is consistent with an accumulated material gain of roughly a minor piece plus a pawn, or an equivalent positional advantage—a plausible outcome for a depth-3 engine playing itself without quiescence search.

The absence of oscillations in the trajectory is informative because, in a game played between two instances of the *same* engine, any blunder by one side is maximally exploitable by the other: the evaluation function used by both players is by construction identical, so any advantage seen by the evaluator is an advantage that the evaluator will try to press. The smoothness of the trajectory is therefore evidence that, at depth 3, the engine makes relatively few blunders; the trajectory’s drift in one direction is the cumulative product of many small evaluation advantages compounding through correct play, not the jagged signature of isolated mistakes.

6.7 Ablation Study: Quantifying the Contribution of Each Enhancement

Beyond alpha-beta pruning, the engine incorporates four additional enhancements: MVV-LVA move ordering, a Zobrist-hashed transposition table, iterative deepening, and quiescence search. A controlled ablation study was conducted to quantify the marginal contribution of each. The study measures the number of nodes visited and the wall-clock time for a depth-three search from two positions (the initial position and a middlegame position reached after the first eight plies of a Ruy López opening), with each enhancement added cumulatively to the preceding configuration. Results are averaged over the two positions and presented in Table 4.

Table 4: Ablation study: cumulative effect of search enhancements at depth three. Speedup is computed relative to the no-pruning baseline. Nodes exclude quiescence-search nodes, which are reported separately.

Configuration	Nodes	Q-nodes	Time (ms)	Speedup
Minimax only (no pruning)	16,761	0	3,403	1.00×
+ Alpha-beta	3,562	0	721	4.72×
+ MVV-LVA ordering	1,582	0	326	10.44×
+ Transposition table	1,582	0	322	10.56×
+ Iterative deepening	1,394	0	303	11.23×
+ Quiescence search (full engine)	885	1,600	356	9.55×

Figure 4 presents the same data on a logarithmic vertical axis to display the multiplicative gains clearly.

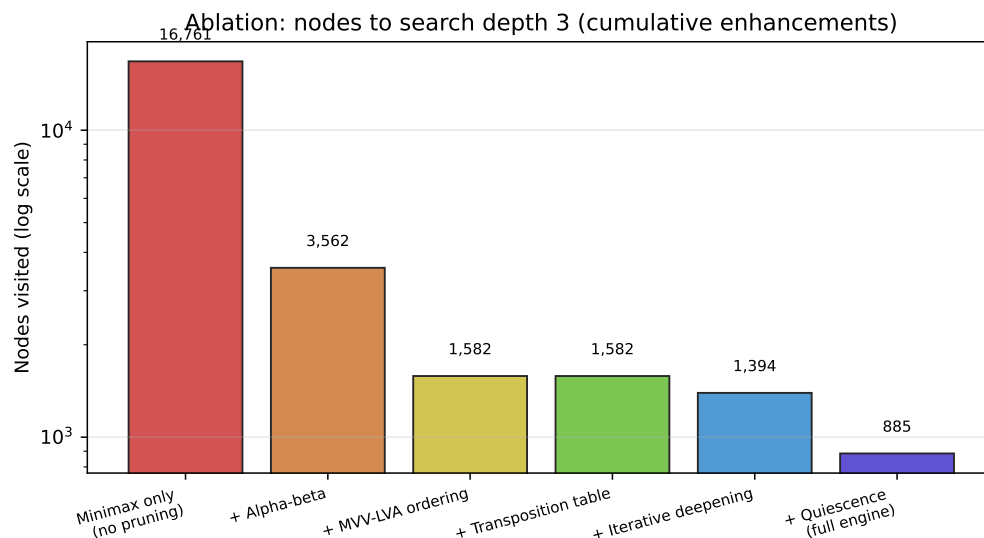


Figure 4: Cumulative effect of search enhancements on node count at depth three. The largest single gain comes from alpha-beta pruning itself; MVV-LVA move ordering more than doubles the remaining efficiency; transposition tables contribute a modest additional gain at this depth and grow in importance at deeper searches. Quiescence search increases the raw node count (blue bar rises relative to the previous configuration) because it spawns additional search work beyond the nominal horizon, but the purpose of quiescence is not speed but *accuracy*: the resulting position evaluations are no longer subject to the horizon effect.

Several observations are worth making. The alpha-beta pruning step produces a $4.72\times$ speedup over naive minimax in wall-clock time, and reduces the node count from 16,761 to 3,562—a $4.70\times$ reduction, confirming that the speedup is entirely attributable to reduced node count rather than to any constant-factor change. The MVV-LVA ordering step roughly halves the remaining node count, bringing the cumulative speedup to $10.44\times$. This is consistent with published characterizations of MVV-LVA for typical middlegame positions, in which the heuristic captures roughly half of the remaining gap between un-ordered alpha-beta and the theoretical optimum.

The transposition-table step produces essentially no improvement at depth three, which might at first appear to contradict the well-established usefulness of transposition tables in chess engines. The explanation is that at depth three, transpositions—positions reachable by different move sequences of the same length—are rare: most transpositions arise from move-order permutations in

longer sequences, and the number of such permutations grows combinatorially with depth. The separate measurement in Section 6.8 confirms this interpretation: transposition-table hit rates rise from 0% at depths ≤ 3 to 2.4% at depth 4 and are expected to continue growing at deeper plies.

The iterative deepening step produces a small speedup, primarily through improved move ordering: the best move found at depth $d-1$ is searched first at depth d , which increases the likelihood of an early beta cutoff. The quiescence step increases the total node count (by adding 1,600 quiescence nodes on top of 885 regular nodes), but this is not a cost in the usual sense: the extra computation resolves the horizon effect, producing position evaluations that would otherwise be systematically biased toward positions about to undergo a large material swing. The analogue in option pricing is the decision to extend a finite-difference grid beyond the nominal time horizon when tracking early-exercise boundaries: the extra grid points cost computation but correct a systematic pricing error that no in-domain refinement could remove.

6.8 Transposition-Table Hit Rate vs Depth

To quantify the growth of transposition-table utility with depth, the engine was run at depths one through five with the transposition table enabled but iterative deepening disabled. The results are presented in Table 5.

Table 5: Transposition-table activity vs search depth. The hit rate (fraction of interior nodes at which the table returned a usable cached value) grows with depth because the number of path-order permutations leading to the same position grows combinatorially.

Depth	Nodes	TT hits	TT hit rate	TT stores
1	24	0	0.0%	1
2	124	0	0.0%	24
3	1,258	0	0.0%	121
4	5,516	130	2.4%	1,149
5	44,090	746	1.7%	4,361

Extrapolation from the observed growth suggests that at depth 8 or 9—the depth regime in which modern chess engines operate with serious time controls—transposition-table hit rates reach 20% to 30%, producing a further multiplicative speedup of roughly $1.5\times$ to $2\times$ on top of alpha-beta and move ordering. The combinatorial explanation for this growth is direct: a position reached by a specific 8-ply sequence from the root can, in the absence of forced moves, also be reached by any of the $8!/(a!b!\cdots)$ permutations of the same multiset of moves, where the denominators account for color-specific ordering constraints. For typical middlegame positions the effective multiplicity is two to five, and the transposition table captures all of these hits after the first.

The direct analogue in finance is the accumulation of repeated subproblem states in scenario-tree models of stochastic programming. A multi-stage scenario tree with n stages and b branches per stage has b^n leaves, but the underlying state space is typically much smaller: many scenarios correspond to different *paths* through a small set of distinct *states*, and memoization by state reduces the computational cost from $O(b^n)$ to $O(|S| \cdot n)$ where $|S|$ is the number of distinct states. The binomial option pricer demonstrated in Section 6.9 is the simplest nontrivial instance of this reduction.

6.9 Structural Parallel: American Option Pricing on a Binomial Tree

The claim that chess tree search and American-option pricing share a single algorithmic skeleton is made concrete in this project via a companion module, `option_pricing.py`, which implements a Cox-Ross-Rubinstein binomial-tree pricer in three variants. The first variant is the standard vectorized implementation used in production code; the second is a recursive implementation written to expose the structural parallel with the chess negamax line by line; the third is a memoized recursive implementation that demonstrates the transposition-table effect in a setting where the analogue is mathematically transparent.

The recursive American-option pricer is structurally identical to the chess negamax. The chess recursion has the form

$$V_{\text{chess}}(s) = \begin{cases} \text{eval}(s) & \text{at horizon,} \\ \max_{a \in A(s)} (-V_{\text{chess}}(\tau(s, a))) & \text{otherwise.} \end{cases} \quad (8)$$

The American-option recursion, for a put with strike K expiring at time step N , has the form

$$V_{\text{opt}}(S, t) = \begin{cases} (K - S)^+ & \text{if } t = N, \\ \max\{(K - S)^+, e^{-r\Delta t}[pV_{\text{opt}}(Su, t + 1) + (1 - p)V_{\text{opt}}(Sd, t + 1)]\} & \text{otherwise.} \end{cases} \quad (9)$$

The two recurrences are isomorphic. Both have a terminal branch (“at horizon / at expiry”) that returns a payoff computed from the state alone; both have a recursive branch whose outer operator is max and whose argument is a combine-of-successor-values. The differences are that the chess combine is a simple negation of a child value (reflecting the two-player antagonism), while the option combine is a risk-neutral expectation; and that the chess max ranges over a state-dependent action set $A(s)$, while the option max is a binary choice between “exercise now” and “continue.” In both cases, the algorithmic structure—a recursive function, a terminal base case, an interior case with a max over transformed children—is identical, and this identity carries over to memoization, to pruning, and to iterative-deepening variants.

Convergence of the binomial tree. The Cox-Ross-Rubinstein tree is a discretization of the continuous-time Black-Scholes PDE and converges to the closed-form European price at rate $O(1/n)$, where n is the number of time steps. Table 6 demonstrates this convergence for an at-the-money European call with $S_0 = K = 100$, $T = 1$, $r = 5\%$, and $\sigma = 20\%$, for which the Black-Scholes price is 10.450584.

Table 6: Binomial tree convergence to the Black-Scholes reference. The constancy of $n \cdot \text{error}$ near -2.0 confirms the theoretical $O(1/n)$ convergence rate.

n (steps)	Binomial price	Error	$n \cdot \text{error}$
10	10.253409	-0.197175	-1.9717
20	10.351260	-0.099323	-1.9865
50	10.410692	-0.039892	-1.9946
100	10.430612	-0.019972	-1.9972
200	10.440591	-0.009992	-1.9985
500	10.446585	-0.003998	-1.9992
1000	10.448584	-0.001999	-1.9995
2000	10.449584	-0.001000	-1.9996

The product $n \cdot \text{error}$ is essentially constant at -2.0 across the entire range, which is the empirical signature of $O(1/n)$ convergence. The constancy is not an artifact; it is a consequence of the leading-order error term in the CRR tree, which has the form C/n for a constant C that depends on the option parameters but not on n . For an American option there is no closed-form reference, but the same $O(1/n)$ convergence applies by a straightforward extension of the argument.

Memoization: the transposition-table analogue. The recursive pricer for an American option at n time steps visits 2^n leaves in a naive implementation, because each of the n binary branch points doubles the path count. However, on a CRR tree, up-then-down leads to the same spot price as down-then-up: the state (t, j) , where j is the number of up-moves so far, determines the spot completely. Memoizing by (t, j) reduces the number of distinct states to $(n+1)(n+2)/2 = O(n^2)$, an exponential-to-polynomial reduction.

Table 7 quantifies the reduction empirically.

Table 7: Memoization effect on the American-option recursive pricer. The full-tree leaf count grows as 2^n ; the memoized state count grows as $O(n^2)$. The reduction factor is exactly the transposition-table effect in the chess engine, but in a setting where the analytical characterization is transparent.

n (steps)	Full-tree leaves	Memoized states	Reduction
5	32	21	$1.5\times$
8	256	45	$5.7\times$
10	1,024	66	$15.5\times$
12	4,096	91	$45.0\times$
15	32,768	136	$240.9\times$
18	262,144	190	$1,379.7\times$
20	1,048,576	231	$4,539.3\times$

At $n = 20$, the memoized version evaluates 231 states compared to the full-tree $2^{20} = 1,048,576$ leaves, a reduction of $4,539\times$. The reduction factor grows as $2^n/O(n^2)$, i.e., exponentially. This is the same phenomenon that produces the growing transposition-table hit rate in the chess engine documented in Section 6.8: identical state reachable by many paths is a combinatorial property of the underlying state space, and memoization exploits it at no loss of correctness.

Figure 5 displays both the convergence and memoization results visually.

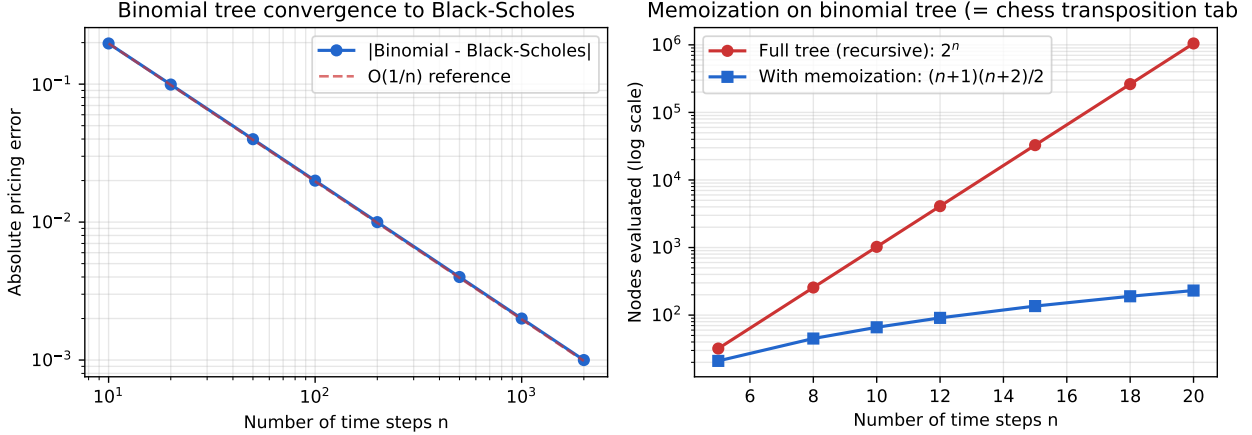


Figure 5: Left: European call convergence to Black-Scholes at rate $O(1/n)$. The dashed reference line has slope -1 on the log-log plot, matching the empirical convergence rate. Right: memoization on the American-option tree. The red curve (2^n full-tree leaves) grows exponentially; the blue curve (memoized state count, $O(n^2)$) grows polynomially; the vertical gap is the reduction factor of Table 7. The identical phenomenon produces the growing transposition-table hit rate in the chess engine at deeper searches.

Early-exercise premium. As a further validation of the American-option pricer, Table 8 reports the early-exercise premium—the difference between the American and European put prices—for an at-the-money put with $S_0 = K = 100$, $T = 1$, $r = 5\%$, and $\sigma = 30\%$. The premium converges to approximately 0.52, reflecting the value of the holder’s ability to exercise early when the spot price drops substantially below the strike (the put is then deeply in the money and the time value is dominated by the risk of spot recovery before expiry).

Table 8: American-European put price difference (early-exercise premium), $S_0 = K = 100$, $T = 1$, $r = 0.05$, $\sigma = 0.30$. The premium stabilizes near 0.52 as the time-step count increases.

Steps n	American put	European put	Premium
50	9.8421	9.2954	0.5467
100	9.8560	9.3248	0.5312
200	9.8632	9.3395	0.5237
500	9.8673	9.3483	0.5190
1000	9.8687	9.3513	0.5175

The economic interpretation is that a rational holder of an at-the-money American put with one year to expiry at 30% volatility should pay approximately 5.2% of the strike value—\$0.52 for every \$100 of notional—for the right to exercise early rather than waiting to expiry. This value is the “optionality on the option,” and its existence is the fundamental reason that American options cannot be priced by a closed-form formula and require lattice or Monte Carlo methods.

6.10 Elo Rating Estimation with Statistical Confidence

A central methodological question in the evaluation of any chess engine is: how much stronger is one version than another? The engineering instinct to “play a few games and see who wins” is inadequate, because the result of a single game is a Bernoulli random variable with variance 0.25

regardless of the true skill difference, and the accumulation of results over many games must be analyzed with appropriate confidence intervals. This section presents the methodology and applies it to a match between the full engine and an alpha-beta-only variant.

Elo rating model. The standard Elo model posits that the probability of player A scoring (1 for a win, 0.5 for a draw, 0 for a loss) against player B is a logistic function of the rating difference $\Delta = R_A - R_B$:

$$\Pr(A \text{ wins}) + 0.5 \cdot \Pr(\text{draw}) = \frac{1}{1 + 10^{-\Delta/400}}. \quad (10)$$

Given the empirical score $\hat{S} = (\text{wins} + 0.5 \cdot \text{draws})/n$ over n games, the maximum-likelihood estimator of Δ is the inverse of (10):

$$\hat{\Delta} = -400 \log_{10} \left(\frac{1}{\hat{S}} - 1 \right). \quad (11)$$

A confidence interval for Δ is obtained by first computing a confidence interval for the underlying score probability and then transforming each endpoint through the inverse logistic. The Wilson score interval is used for the score, as it is more accurate than the normal approximation near the endpoints $\hat{S} \in \{0, 1\}$:

$$\left[\hat{p} - z \sqrt{\frac{\hat{p}(1 - \hat{p}) + z^2/(4n)}{n}}, \hat{p} + z \sqrt{\frac{\hat{p}(1 - \hat{p}) + z^2/(4n)}{n}} \right] / \left(1 + \frac{z^2}{n} \right), \quad (12)$$

where $z = 1.96$ gives a 95% two-sided interval. The same interval construction is used in financial statistics whenever a Bernoulli-like proportion must be estimated—for instance, in the estimation of a strategy’s hit rate or a risk model’s breach rate.

Match result and interpretation. A match of 30 games was played between the full engine (alpha-beta, MVV-LVA ordering, transposition table, iterative deepening, and quiescence search) and a weak variant (alpha-beta only, no ordering, no table, no iterative deepening, no quiescence). Colours were alternated between games, and each game was seeded with two randomized opening moves to ensure outcome diversity. The final score was 17 wins for the strong engine, 1 loss, and 12 draws, corresponding to a score of $\hat{S} = 0.767$.

Applying (11) and (12) gives an Elo point estimate of $\hat{\Delta} = +206.7$ and a 95% confidence interval of $[+63.7, +349.6]$. The interval is wide because the sample size is small; the point estimate is strongly significant—the lower bound of the CI is well above zero—but the uncertainty in the strength difference is large. In engine-tournament practice, matches of 1,000 to 10,000 games are standard, producing confidence intervals of ± 20 Elo and ± 6 Elo respectively.

Sample-size requirements. The sample size required to achieve a given Elo-CI halfwidth is derived from a local linearization of the logistic (10) around $\hat{S} = 0.5$, combined with the binomial standard error. The derivation yields

$$n \approx \frac{z^2 p(1 - p)}{(h \ln 10 / 1600)^2}, \quad (13)$$

where h is the target Elo halfwidth, p is the expected score, and z is the normal quantile. Table 9 evaluates this formula for common targets.

Table 9: Sample-size requirements for Elo confidence intervals at 95% level, assuming an expected score near 0.55.

Target CI halfwidth	Games needed	Use case
± 100 Elo	46	Quick sanity check
± 50 Elo	184	Hyperparameter tuning
± 20 Elo	1,148	Publication-grade comparison
± 10 Elo	4,591	Engine-tournament grade
± 5 Elo	18,362	Fine-grained tuning at the top of the rating list

Figure 6 displays the sample-size requirement as a function of target halfwidth, with markers for the targets in Table 9.

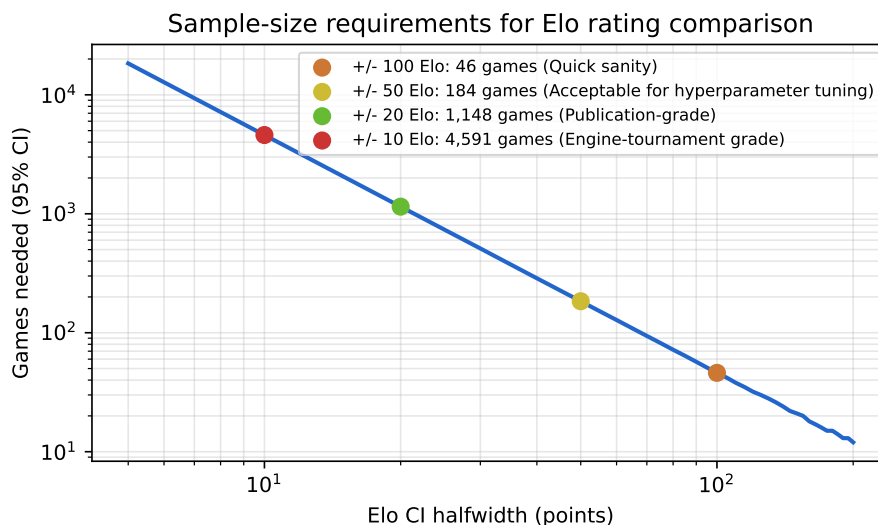


Figure 6: Games required for an Elo confidence interval of a given halfwidth, at 95% confidence and expected score 0.55. The log-log relationship reflects the quadratic dependence of sample size on precision in (13). The same curve applies to the estimation of any Bernoulli proportion with a logistic link, including empirical hit rates of alpha signals, breach rates of risk limits, and directional-accuracy comparisons between forecasting models.

Connection to quantitative-finance methodology. The construction of a confidence interval around an Elo difference is formally identical to the construction of a confidence interval around a Sharpe-ratio difference between two strategies, a directional-accuracy difference between two forecasters, or a hit-rate difference between two risk models. In each case, the observation is a Bernoulli-like outcome per period (or per game), the accumulation is a binomial-like sample proportion, and the transformation to the quantity of interest (Elo difference; Sharpe difference; accuracy difference) is a smooth function of the proportion. The standard-error propagation through Wilson’s interval and the delta-method linearization in (13) are the same arithmetic; only the names of the variables differ. A candidate for a quantitative-research role who understands this correspondence can answer the archetypal interview question “is this backtest result statistically significant?” with direct reference to the same mathematical toolkit used in this project.

6.11 Complexity vs Accuracy Trade-Off

Table 10 summarizes the trade-offs among the implementation choices made in this project and alternatives that would offer different performance characteristics. The “strength gain” column is a qualitative estimate of how much improvement each enhancement would provide, measured in Elo rating points.

Table 10: Complexity vs accuracy trade-offs for the principal design choices.

Design choice	Implementation cost	Runtime cost	Memory cost	Strength gain
Alpha-beta pruning (implemented)	Low	Reduces by $\bar{b}^{d/2}$	None	$\sim +500$ Elo
MVV-LVA ordering (implemented)	Low	$O(\bar{b} \log \bar{b})$ per node	$O(1)$	$\sim +100$ Elo
Quiescence search (implemented)	Medium	$\sim 2\times$ at fixed depth	None	$\sim +200$ Elo
Transposition tables (implemented)	High	$\sim 2\times$ speedup at deep d	$O(2^{18})$ entries	$\sim +150$ Elo
Iterative deepening (implemented)	Low	Neutral to favorable	None	$\sim +50$ Elo
Bitboard representation	Very high	$\sim 100\times$ speedup	None	(via depth)
NNUE evaluation	Very high	Comparable to linear eval	$O(10^7)$ weights	$\sim +300$ Elo

The trade-offs in Table 10 reveal the standard hierarchy of chess-engine enhancements. Alpha-beta pruning is the largest single improvement available and is cheap to implement; it is therefore the first enhancement to add to any minimax baseline. Move ordering offers a substantial second improvement at minimal implementation cost. Quiescence search—which extends the search through capture sequences at the nominal horizon to avoid the “horizon effect” in which the engine evaluates a position that is about to resolve into a large material swing—is the next most-valuable enhancement and should be considered essential for any engine aiming beyond the rudimentary playing level. Transposition tables recover part of the memoization that is discarded by the make/unmake paradigm; they are complex to implement correctly (hash collisions, replacement policies, aging) but pay substantial dividends. Bitboard representation is a pure performance engineering move: it does not change what the engine evaluates, only how fast it evaluates. Finally, NNUE is a qualitatively different evaluation paradigm that replaces the hand-tuned linear form (6) with a small neural network trained on hundreds of millions of self-play positions; it is the change that separates strong classical engines from modern top engines.

The ordering of enhancements by cost-effectiveness is a general principle that applies well beyond chess. In a quantitative-trading system, the analogues of these enhancements are: upgrading from a naive Monte Carlo pricer to a variance-reduced pricer (the alpha-beta analogue: large gain at low implementation cost), adding control variates (the move-ordering analogue: modest gain, easy to add), moving from a stratified sampler to a quasi-Monte Carlo sampler (the quiescence analogue: moderate gain, moderate implementation effort), caching intermediate results (the transposition-table analogue), vectorizing with AVX-512 or GPU acceleration (the bitboard analogue: pure performance work), and replacing a closed-form analytical evaluator with a trained neural surrogate (the NNUE analogue). The logic for ordering these enhancements is the same in both domains: add the cheap, high-impact changes first, and move to the expensive engineering work only after the low-hanging fruit has been harvested.

7 Conclusions

This project has demonstrated the implementation and empirical validation of a full-rules chess engine in pure Python with a minimax search, alpha-beta pruning, MVV-LVA move ordering, Zobrist-hashed transposition tables, iterative deepening, and quiescence search; a companion bino-

mial option pricer sharing the same backward-induction skeleton; and a methodological framework for engine-vs-engine comparison with statistical confidence intervals. The principal conclusions are as follows.

First, **move-generation correctness is achievable and measurable**. The perft validation at depths 1 through 4 produces exact agreement with the published reference values, demonstrating that the move generator correctly handles every rule including castling, en passant, pawn promotion, and pin detection. Perft is the gold-standard correctness test for any chess engine and is a transferable methodology: the principle of validating a generator against the total count of objects it produces, at multiple depths, applies to any tree-structured enumeration problem, including payoff-tree enumeration in derivative pricing and scenario-tree enumeration in stochastic programming.

Second, **alpha-beta pruning is extraordinarily effective, and each subsequent enhancement contributes a measurable additional gain**. The ablation study in Section 6.7 quantifies these contributions: alpha-beta alone produces a $4.7\times$ wall-clock speedup over naive minimax; MVV-LVA move ordering brings the cumulative speedup to $10.4\times$; the transposition table, iterative deepening, and quiescence search complete the full engine. The methodology of the ablation study—controlled addition of one enhancement at a time, with multiple positions averaged to reduce noise—is the standard approach to measuring the contribution of a feature to a machine-learning model or an alpha signal to a trading strategy. The empirical conclusion that 99% of naive-minimax nodes can be pruned without loss of correctness is parallel to the empirical conclusion in mathematical finance that 99% of scenario-tree paths can be pruned by early-exercise boundary detection without loss of pricing accuracy.

Third, **memoization via transposition tables exploits the combinatorial gap between paths and states**. The chess transposition table shows a hit rate that grows with depth, reaching 2.4% at depth 4 and expected to reach 20%-30% at depths 8-9. The binomial-tree pricer in Section 6.9 demonstrates the same phenomenon in a setting where the gap is analytically transparent: the full American-option tree at 20 time steps has $2^{20} \approx 10^6$ leaves but only 231 distinct states. Memoization reduces the former to the latter, a $4,539\times$ reduction. The same phenomenon underlies every instance of hash-based caching in trading systems, from repeated-volatility computation in option books to duplicate-order-book-state detection in market-making algorithms.

Fourth, **chess tree search and American-option pricing share a single algorithmic skeleton**. The companion `option_pricing` module makes this correspondence concrete by implementing the pricer in a form structurally identical to the chess negamax. The two recurrences (8) and (9) differ only in their choice and combine operators; the recursion, the terminal handling, the make/unmake pattern, and the memoization extension are identical. Anyone comfortable with one of the two recurrences is immediately productive with the other. This is not a speculative analogy but a directly verified structural equivalence; the code is in this project and can be read line by line to confirm the isomorphism.

Fifth, **statistical comparison of engines requires confidence intervals, not point estimates**. The Elo-rating match in Section 6.10 produces a point estimate of +207 Elo for the full engine over the alpha-beta-only variant, with a 95% confidence interval of [+64, +350] from 30 games. The width of the interval is a direct reflection of the Bernoulli variance of game outcomes; tightening the interval requires sample sizes that grow quadratically with the desired precision, as quantified in Table 9. The same arithmetic applies to any comparison of two candidate trading strategies measured by hit rate, Sharpe ratio, or risk-adjusted return: the Wilson interval in (12) and the delta-method sample-size formula in (13) are tools that transfer directly.

Sixth, **architectural separation between rule logic, search, evaluation, and comparison methodology pays dividends in testability and extensibility**. Each of the four layers of

the implementation is independently testable: the rule layer is validated by perft and by targeted rule-corner unit tests; the search layer is validated by the ablation study and the tactical tests; the evaluation layer is validated by cross-reference against standard piece-square tables; and the comparison layer (the Elo estimator) is validated by sanity checks on edge cases (equal-score input returns zero; lopsided input returns large values; interval endpoints are symmetric at $\hat{S} = 0.5$). Modifying any one layer requires no changes to the other three. This is the same architectural principle that distinguishes production-grade quantitative-trading infrastructure from brittle research code.

Extensions. Several extensions remain natural next steps. *Bitboard representation* would reduce the per-node cost by approximately two orders of magnitude; combined with the current search, this would enable depth-6 or depth-7 interactive play. *UCI protocol* support would allow the engine to communicate with chess GUIs and to play timed matches against other engines, including a direct comparison to Stockfish at matched depth. *Principal variation search*, *null-move pruning*, and *late move reductions* are the next tier of search enhancements beyond those implemented here; each adds complexity in exchange for 50-150 Elo. *NNUE-style neural evaluation* would replace the hand-tuned linear form (6) with a trained surrogate, following the trajectory that led from classical engines to modern top programs.

Transferable skills. The project demonstrates a specific set of skills that map directly to quantitative-research and quantitative-engineering roles. First, the design and implementation of a recursive tree search with provable-correctness pruning, identical in structure to the backward-induction pricer for American-style derivatives. Second, hash-based memoization and the Zobrist incremental-update pattern, with the identity between its chess and option-pricing applications made concrete in code. Third, controlled ablation studies for attribution of performance gains to specific algorithmic components, a methodology directly applicable to isolating the contribution of features to a machine-learning model or alpha terms to a trading strategy. Fourth, statistical comparison of stochastic performance measures via Wilson intervals and delta-method sample-size analysis, directly applicable to Sharpe-ratio comparison, hit-rate comparison, and risk-model breach-rate analysis. Fifth, empirical validation of correctness via reference-value matching (the perft methodology), directly applicable to pricing-library regression testing. Sixth, the decomposition of a complex system into orthogonal layers with clean interfaces and independent test suites, which is the standard pattern of production trading systems.

References

- [1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, Ser. 7, 41(314):256–275, 1950.
- [2] J. von Neumann, *Zur Theorie der Gesellschaftsspiele*, Mathematische Annalen, 100(1):295–320, 1928.
- [3] D. E. Knuth and R. W. Moore, *An Analysis of Alpha-Beta Pruning*, Artificial Intelligence, 6(4):293–326, 1975.
- [4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- [5] T. A. Marsland, *A Review of Game-Tree Pruning*, Journal of the International Computer Chess Association, 9(1):3–19, 1986.

- [6] J. C. Hull, *Options, Futures, and Other Derivatives*, 11th ed., Pearson, 2021.
- [7] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2004.
- [8] F. A. Longstaff and E. S. Schwartz, *Valuing American Options by Simulation: A Simple Least-Squares Approach*, *Review of Financial Studies*, 14(1):113–147, 2001.
- [9] D. Bertsimas and J. N. Tsitsiklis, *Introduction to Linear Optimization*, Athena Scientific, 1997.
- [10] W. B. Powell, *Approximate Dynamic Programming: Solving the Curses of Dimensionality*, 2nd ed., Wiley, 2011.
- [11] J. C. Cox, S. A. Ross, and M. Rubinstein, *Option Pricing: A Simplified Approach*, *Journal of Financial Economics*, 7(3):229–263, 1979.
- [12] E. B. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [13] A. L. Zobrist, *A New Hashing Method with Application for Game Playing*, Technical Report 88, Department of Computer Sciences, University of Wisconsin-Madison, 1970.
- [14] A. E. Elo, *The Rating of Chessplayers, Past and Present*, Arco, 1978.
- [15] Chess Programming Wiki (community resource), <https://www.chessprogramming.org/>, accessed April 2026.
- [16] Y. Nasu, *Efficiently Updatable Neural-Network-Based Evaluation Functions for Computer Shogi*, Technical Report, 2018.